ЛАБОРАТОРНАЯ 2 · ВАРИАНТ «СВЕТОФОР»

UART и прерывания простыми словами

Как управлять светофором с компьютера, чем отличаются опрос и прерывания и что отвечать на защите.

Что добавилось после первой лабораторной

Светофор по-прежнему переключает лампы и реагирует на кнопку. Теперь ему можно отправить текстовую команду с компьютера: узнать состояние, выключить обработку кнопки или изменить длительность красного.

Для обмена используется **UART** - устройство внутри микроконтроллера, которое передаёт и принимает данные по одному биту. Подробное объяснение начинается со следующей страницы.

Что можно сказать в начале защиты

«Я добавил к светофору управление с компьютера по UART. Команда собирается из полученных символов и выполняется после Enter. Для временного хранения данных есть очереди. Обмен работает двумя способами: через опрос и через прерывания. Между ними можно переключаться командой. Ещё есть две сборки: с готовыми функциями HAL и с моим собственным UART-драйвером. В обоих случаях правила команд одинаковые».

Как пользоваться памяткой

Сначала прочитай объяснения. Затем закрой ответы и попробуй отвечать своими словами. Вопросы 1-50 дают основу, 51-80 помогают разобрать детали кода. В конце есть примеры, план демонстрации и короткая самопроверка.

Памятка составлена по текущему коду lab2, его документации и разделу 5.2 пособия ИТМО. Это подготовка по твоей реализации; официальный список вопросов преподавателя не предоставлен.

1. Как команда попадает в микроконтроллер

На компьютере ты открываешь **терминал** - программу, в которой можно печатать текст и видеть ответы устройства. Например, отправляешь ?, чтобы узнать состояние светофора.



Команда идёт к плате, ответ возвращается в терминал.

Что делает FT2232

На стенде есть микросхема FT2232. В этой работе она помогает связать USB компьютера с UART микроконтроллера. Поэтому компьютер видит удобный для терминала последовательный порт.

TX и RX

TX - линия передачи, **RX** - линия приёма. С точки зрения микроконтроллера TX отправляет ответ, а RX принимает команду. Это два разных направления, поэтому приём и передача могут идти одновременно.

На микроконтроллере	Назначение
PA9, USART1 TX	Ответы от платы к компьютеру
PA10, USART1 RX	Команды от компьютера к плате
AF7	Режим выводов, который подключает их к USART1

USART1 - название используемого блока в STM32. Он умеет больше, чем нужно здесь. В этой работе включён обычный асинхронный обмен UART: отдельного провода тактовых импульсов нет.

Если соединять два UART вручную

Передачу одного устройства соединяют с приёмом другого: TX с RX. Нужна общая земля и совместимые электрические уровни. На учебном стенде нужные связи уже есть в схеме; дополнительные провода для DBG USB не требуются.

2. Что означает 57600 8N1

Два устройства должны договориться, как передавать данные. Эти настройки у платы и терминала должны совпадать.

Настройка	Простой смысл
57600	Скорость линии: номинально 57600 бит в секунду
8	В одном байте данных восемь бит
N	Дополнительный бит проверки чётности не передаётся
1	Один стоп-бит после данных
Flow control: None	Нет отдельного управления потоком передачи

Как выглядит передача одного символа

Для простых латинских букв и знаков один символ обычно занимает один байт. Например, ? передаётся одним байтом. Байт состоит из восьми нулей и единиц - **битов**.

```
старт  |  8 бит данных  |  стоп
   0   |  b0 ... b7     |   1
```

Когда данных нет, линия находится в единице. Стартовый ноль сообщает о начале байта. Затем идут восемь бит данных, начиная с младшего, и стоповая единица. Этот набор называют **кадром**.

Почему 57600 - это не 57600 букв в секунду

На один байт данных в формате 8N1 уходит 10 бит линии: один стартовый, восемь данных и один стоповый. Значит, теоретический предел - около 5760 байт в секунду, если передавать без пауз.

Один такой кадр занимает примерно 174 микросекунды. Поэтому даже короткое текстовое сообщение передаётся не мгновенно.

Зачем заранее задавать одинаковую скорость

У UART здесь нет общего провода с тактовыми импульсами. Приёмник сам отсчитывает моменты чтения битов. Если скорости сильно различаются, символы могут искажаться или появятся ошибки приёма.

3. Как подключить терминал

Какой разъём использовать

Нужен **DBG USB**. У FT2232 два интерфейса: Interface 0 служит для прошивки и отладки, Interface 1 - для обмена UART. Это разные задачи, хотя они используют одну микросхему.

В Windows для Interface 0 в этом проекте нужен WinUSB, для Interface 1 - драйвер виртуального COM-порта FTDI VCP. COM-порт - имя канала, который открывается в терминале.

Настройки терминала

Параметр	Значение
Порт	UART-канал FT2232, то есть Interface 1
Скорость	57600
Биты данных	8
Чётность	None
Стоп-биты	1
Flow control	None
Local echo	Off

Что такое эхо

Плата отправляет полученный символ обратно. Ты набрал ?, плата вернула ?, и он появился в окне терминала. Это помогает видеть, что именно устройство получило.

Local echo - похожая функция самого терминала. Её выключают, чтобы символ не отображался дважды: сначала силами терминала, затем после ответа платы.

Что увидеть после сброса

```
Lab2 traffic: 57600 8N1, mode 1, timeout 12, I
```

Затем введи ? и нажми Enter. Должен прийти ответ о состоянии. Название COM-порта зависит от компьютера; в Linux обычно используется имя вида `/dev/ttyUSB...`.

4. Команды: что вводить и что получится

Каждая команда выполняется после **Enter**. Пиши строчными буквами и с пробелами, как в таблице. В реализации регистр букв и лишние пробелы имеют значение.

Команда	Что делает
?	Показывает фазу, режим кнопки, длительность красного и способ обмена
set mode 1	Разрешает учитывать нажатия кнопки
set mode 2	Игнорирует кнопку и убирает уже сохранённую просьбу
set timeout 8	Устанавливает обычный красный на 8 секунд
set interrupts on	Включает UART с прерываниями: I
set interrupts off	Включает UART с опросом: P

Как прочитать ответ на ?

```
red, mode 1, timeout 12, I
```

Это значит: сейчас красный; кнопка учитывается; обычный красный настроен на 12 секунд; UART работает по прерываниям.

Названия фаз: `red` - красный, `green` - обычный зелёный, `blinking green` - мигание, `yellow` - жёлтый. При мигании имя фазы не меняется в те полсекунды, когда лампа выключена.

Что значит timeout в этой работе

Это **длительность красного** в секундах. Допустимо целое число от 1 до 86400. Например, `set timeout 1` задаёт одну секунду, а не одну миллисекунду. Зелёный, мигание и жёлтый эта команда не меняет.

Ответы

Успешная настройка возвращает `OK`. Неизвестная команда или неверное число дают сообщение об ошибке. Пустой Enter ничего не выполняет. Программа принимает окончания строки CR, LF и их пару CRLF.

5. Три настройки, которые легко перепутать

В работе есть **три независимых выбора**. Их полезно сразу различать.

Что выбираем	Варианты	Как меняем
Учитывать ли кнопку	mode 1 / mode 2	Командой во время работы
Как обслуживать UART	I: прерывания / P: опрос	Командой во время работы
Чьим кодом управлять UART	HAL / собственный драйвер	Выбором сборки до прошивки

Пример с кнопкой

Выполнили `set mode 2`. Кнопка перестала сокращать красный. При этом UART может продолжать работать через прерывания. Светофор тоже продолжает переключаться.

Пример с UART

Выполнили `set interrupts off`. Стало P: программа сама проверяет UART в основном цикле. Это не выключает кнопку и не меняет длительность красного.

Пример с выбором драйвера

Собрали собственную версию и прошили её. В ней тоже можно выбирать I или P командой. Команда `set interrupts on` выбирает способ обмена, но не заменяет собственный код библиотекой HAL.

Что включено после сброса

Красный начинается заново. Обычный красный - 12 секунд. Кнопка разрешена: mode 1. UART работает по прерываниям: I.

Короткий ответ для защиты

«Mode 1/2 относится к кнопке. I/P относится к UART. HAL или собственный драйвер выбирается при сборке. Это три разные настройки».

6. Опрос и прерывания на простом примере

Опрос: программа сама регулярно проверяет UART

Представь, что ты время от времени смотришь, пришло ли сообщение. Это **опрос**, или polling. Главный цикл спрашивает UART: «Есть новый байт? Можно отправить следующий?» Затем продолжает другие дела.

Прерывание: устройство сообщает о событии

Это похоже на сигнал о новом сообщении. Когда разрешённое событие UART произошло, процессор ненадолго переходит в специальную функцию. Она быстро обслуживает UART, после чего основной код продолжает работу.

Эта функция называется **обработчиком прерывания**. В коде её имя - `USART1_IRQHandler`. Она не разбирает длинную команду: байт сохраняется, а смысл команды выясняет основной цикл.

Кто разрешает прерывания

Нужно разрешить нужное событие в самом USART и разрешить его обработчик в **NVIC**. NVIC - часть процессора, которая выбирает, какое прерывание обслужить и с каким приоритетом.

Приоритет показывает, какое прерывание важнее. В этой настройке меньшее число означает более высокий приоритет: у SysTick он 0, у USART1 - 1. Поэтому отсчёт времени может прервать обработку UART.

Что означает «неблокирующая функция»

Функция быстро заканчивает работу и не ждёт, пока весь обмен завершится на проводе. Например, помещает текст в очередь и возвращает управление. Так светофор может продолжать работать во время передачи.

В режиме I операции UART не ждут окончания кадра. В режиме P приём тоже не ждёт. Но передача одного байта через HAL в P может немного подождать: в ней задан тайм-аут 2 мс. Собственная версия в P только проверяет готовность и возвращается.

Важная оговорка

`set interrupts off` выключает прерывания **UART**, а не вообще все прерывания. SysTick продолжает отсчитывать время для светофора.

7. Зачем нужны очереди

UART передаёт байты по одному. Основная программа в это время занимается кнопкой, временем и командами. Чтобы данные могли немного подождать, в памяти есть **буферы** - места временного хранения.

RX и TX

Буфер	Что в нём хранится	Размер по умолчанию
RX	Принятые байты, которые программа ещё не разобрала	128 байт
TX	Байты, которые нужно передать компьютеру	512 байт

Очередь сохраняет порядок: первым выходит то, что пришло первым. Текст 0K должен уйти именно как O, затем K, а не наоборот.

Почему буфер кольцевой

Представь пронумерованные ячейки по кругу. Дошли до последней - продолжаем с первой, если она уже свободна. Данные не приходится каждый раз сдвигать к началу массива.

В коде head указывает, куда записывать, `tail` - откуда читать, `count` хранит количество занятых ячеек.

Что делает serial_write

Пытается положить весь переданный текст в TX. Если места достаточно, возвращает true. Если места мало, возвращает false и не кладёт даже часть этого текста.

Успех serial_write ещё не означает, что ответ уже дошёл до компьютера. Он означает только, что байты приняты в очередь отправки.

Если компьютер шлёт слишком много

Любой буфер конечный. Здесь нет отдельного управления потоком, поэтому бесконечно быстрый ввод без потерь не гарантируется. Для демонстрации отправляй следующую команду после ответа на предыдущую.

Когда TX занят, консоль временно приостанавливает разбор входа. Она заранее оставляет место под эхо и полный ответ. Светофор в это время продолжает обслуживаться.

8. Как из символов получается команда

Сначала собрать строку

Байты приходят по одному: например, сначала `s`, потом `e`, потом `t`. Консоль сохраняет их в строке. Enter означает: «Команда закончилась, её можно проверить».

В строке допускается максимум **63 символа**. Буфер содержит 64 ячейки; последняя нужна для специального нуля, которым в C отмечают конец строки.

CR и LF - два варианта символа окончания строки. Если терминал присылает их парой CRLF, программа воспринимает пару как один Enter, чтобы не выполнять команду дважды.

Потом проверить смысл

Функция `command_parse` проверяет, совпадает ли строка с известной командой. Такой разбор текста называют **парсингом**. Например, после `set timeout` должны идти только цифры допустимого числа.

Ввод	Результат
set timeout 8	Допустимо: обычный красный 8 с
set timeout 0	Ошибка: ноль запрещён
set timeout -1	Ошибка: знак минус запрещён
set timeout 8abc	Ошибка: после числа есть посторонние буквы
set interrupts ON	Ошибка: ожидается on маленькими буквами

Зачем отбрасывать повреждённую строку целиком

Если байт потерян, прежняя команда может превратиться в другой текст. Выполнять её опасно для правильности программы. Поэтому при ошибке приёма или переполнении текущая строка отвергается до следующего Enter.

После Enter приходит просьба повторить команду. Затем можно отправить её заново. Слишком длинная строка обрабатывается так же.

Редактирование

Backspace и Delete убирают последний символ из ещё не выполненной команды. Они меняют собранный текст; как удаление выглядит в окне, зависит от терминала.

9. RXNE, TXE и TC: что уже готово

UART показывает своё состояние специальными битами - **флагами**. Флаг равен единице, когда произошло соответствующее событие.

Флаг	Что он сообщает
RXNE	Пришёл байт, его можно прочитать из DR
TXE	Регистр передачи DR свободен и может принять новый байт
TC	Последний кадр целиком ушёл по линии

Почему TXE и TC - разные вещи

Можно представить конвейер: сначала байт кладут в ячейку DR, потом устройство забирает его и постепенно выдаёт биты на провод. Ячейка уже освободилась, но биты ещё передаются.

TXE: «Можно положить следующий байт». TC: «Передача последнего байта закончилась». Перед сменой I/P программа ждёт именно TC.

Основные регистры

Регистр	Зачем нужен
SR	Показывает флаги готовности и ошибок
DR	Через него читают принятый байт и записывают передаваемый
BRR	Задаёт делитель для скорости UART
CR1, CR2, CR3	Хранят настройки обмена и разрешения событий

Откуда берётся BRR = 278

USART1 получает 16 миллионов тактов в секунду. Для выбранного здесь режима делим 16000000 на 57600 и округляем: получается 278. Это значение записано как `0x116` в шестнадцатеричном виде.

С таким делителем расчётная скорость около 57554 бит/с. Она немного отличается от 57600: ошибка делителя примерно 0,08%. К этому добавляется погрешность внутреннего генератора.

10. Что делает каждая часть программы

Файл	Простой смысл
main.c	Повторяет обслуживание UART, команд, кнопки и светофора
serial.c	Хранит очереди RX/TX и следит за отправкой
console.c	Собирает строку, делает эхо и формирует ответ
command.c	Проверяет, какую команду ввели
traffic.c	Хранит фазу и настройки светофора
uart_hal.c	Общается с UART через готовые функции HAL
uart_register.c	Общается с UART напрямую через регистры
uart_pins.c	Подключает PA9/PA10 к USART1

Путь команды set timeout 8

1. UART получает байты. Они попадают в RX. 2. Консоль собирает строку до Enter. 3. Разборщик узнаёт команду и число 8. 4. Светофор сохраняет длительность 8000 мс. 5. Ответ OK попадает в TX и уходит на компьютер.

Как безопасно меняется I/P

После команды программа сначала заканчивает отправку уже поставленного текста. Она ждёт пустой TX, отсутствия занятой передачи и флага TC. При этом продолжает другие дела - отдельного бесконечного ожидания нет.

Затем меняет режим UART. Эхо команды успевает уйти старым способом, а **OK отправляется уже новым способом**. Очередь принятых байтов сохраняется. Следующая команда ждёт завершения переключения.

Почему обработчик короткий

Если разбирать длинные команды прямо в прерывании, основной код будет дольше ждать. Здесь обработчик принимает или отправляет байт и возвращает управление. Текст разбирается в основном цикле порциями до 16 символов.

Вопросы 1-10. Как устроен обмен

Короткие ответы, которые можно произнести на защите своими словами.

1. Что добавилось в ЛР 2?

Управление светофором с компьютера по UART: можно узнавать состояние, менять красный и включать или отключать обработку кнопки.

2. Что такое UART?

Устройство для последовательного обмена: оно передаёт и принимает данные по одному биту. В этой работе отдельного тактового провода нет.

3. Почему в коде USART1, если работа про UART?

Блок USART умеет и синхронный, и асинхронный обмен. Здесь используется его асинхронный режим, то есть обычный UART.

4. Для чего TX и RX?

TX передаёт данные от устройства, RX принимает. Передачу одного UART подключают к приёму другого.

5. Какие выводы используются?

PA9 - TX микроконтроллера, PA10 - RX. Оба подключены к USART1 через AF7.

6. Что означает 57600?

Номинальную скорость в битах в секунду. Это не число букв: вместе с данными передаются служебные биты.

7. Что означает 8N1?

Восемь бит данных, без проверки чётности, один стоп-бит. Перед данными также идёт стартовый бит.

8. Зачем стартовый и стоповый биты?

Они обозначают начало и конец кадра. Приёмник использует старт для отсчёта моментов чтения данных.

9. Сколько бит линии занимает один байт в 8N1?

Десять: один стартовый, восемь данных и один стоповый. Поэтому предел без пауз около 5760 байт в секунду.

10. Можно ли одновременно принимать и передавать?

Да. Здесь есть отдельные линии TX и RX, и включены передатчик с приёмником. Такой обмен называют полнодуплексным.

Вопросы 11-20. Терминал и команды

11. Как компьютер связан с UART платы?

Через FT2232 на стенде и разъём DBG USB. UART доступен через виртуальный COM-порт Interface 1.

12. Какие настройки поставить в терминале?

57600, 8 бит данных, без чётности, один стоп-бит, без flow control. Local echo выключить.

13. Что такое эхо?

Плата возвращает принятый символ. Так видно, какой текст дошёл до устройства.

14. Почему символы могут появляться дважды?

Может быть включён Local echo в терминале. Тогда символ показывает сам терминал, а затем ещё раз - после возврата от платы.

15. Когда выполняется команда?

После Enter. До этого консоль только собирает текст. Здесь принимаются CR, LF и их сочетание CRLF.

16. Что возвращает вопросительный знак?

Текущую фазу, режим кнопки, заданную длительность красного и букву I или P - способ работы UART.

17. Что делают set mode 1 и set mode 2?

Mode 1 разрешает кнопку. Mode 2 игнорирует нажатия и убирает уже сохранённую просьбу сократить красный.

18. Что делает set timeout X?

Устанавливает длительность обычного красного в секундах. X - целое число от 1 до 86400.

19. Что делают set interrupts on и off?

On выбирает UART с прерываниями, то есть I. Off выбирает опрос, то есть P. Это не режим кнопки.

20. Что получится при неправильной команде?

Программа вернёт сообщение об ошибке и не применит неверный параметр. Регистр букв и пробелы здесь значимы.

Вопросы 21-30. Что меняется у светофора

21. Что включено после сброса?

Красный, режим кнопки 1, тайм-аут 12 секунд и UART с прерываниями I.

22. Команда mode 2 останавливает светофор?

Нет. Она выключает только влияние кнопки. Цвета продолжают переключаться по времени.

23. Команда timeout меняет время всех цветов?

Нет. Меняется красный. Зелёный остаётся 3 секунды, мигание 3 секунды, жёлтый 1 секунду.

24. Если задать timeout 8, сколько длится короткий красный?

Две секунды: четверть от восьми. Они отсчитываются от начала красного, а не от момента нажатия.

25. При timeout 8 нажали через секунду красного. Что будет?

После подтверждения нажатия останется около секунды: общий красный должен занять две секунды.

26. При timeout 8 нажали через пять секунд красного. Что будет?

Зелёный включится сразу после подтверждения. Порог короткого красного в две секунды уже пройден.

27. Смена timeout заново запускает красный?

Нет. Время начала фазы сохраняется. Если новый порог уже пройден, ближайшая проверка переключит фазу.

28. Что будет с сохранённой просьбой после mode 2?

Она стирается. Если вернуться в mode 1, старая просьба автоматически не восстановится.

29. При timeout 1 короткий красный равен нулю?

Нет, он равен 250 мс. В программе сначала секунды переводятся в миллисекунды, затем длительность делится на четыре.

30. Если держать кнопку в mode 2 и включить mode 1, появится нажатие?

Само включение режима его не создаст. Проверка кнопки продолжалась всё время; для нового события нужно отпустить её и нажать снова.

Вопросы 31-40. Опрос и прерывания

31. Что такое опрос UART?

Основной цикл регулярно проверяет готовность UART и выполняет доступное действие. Устройство не вызывает свой обработчик прерывания.

32. Что такое прерывание?

Короткий переход процессора к специальной функции при событии. После её выполнения основной код продолжает работу.

33. Что такое NVIC?

Контроллер прерываний внутри процессора. Он разрешает обработку и учитывает приоритеты. Здесь меньшее число означает более высокий приоритет: SysTick 0, USART1 1.

34. Почему мало разрешить только NVIC?

Нужное событие должно быть разрешено и в самом USART. Источник события и его обработка процессором настраиваются отдельно.

35. Что значит I и P?

I - interrupt, обмен с прерываниями. P - polling, обмен через опрос.

36. Выключаются ли все прерывания командой off?

Нет. Отключаются прерывания UART. SysTick продолжает считать время для светофора.

37. Где разбирается текст команды?

В основном цикле через console_update и command_parse. Обработчик UART занимается приёмом, отправкой байта и сообщением об ошибке.

38. Что означает неблокирующая отправка?

Функция не ждёт, пока весь текст уйдёт по проводу. Она ставит байты в очередь и возвращается; дальше обмен обслуживается отдельно.

39. В режиме опроса функции совсем никогда не ждут?

Приём не ждёт. Но HAL-передача одного байта в P имеет тайм-аут 2 мс. Собственная передача в P только проверяет готовность.

40. Почему при обмене продолжается работа светофора?

Основной цикл выполняет короткие шаги и не ждёт полного ответа на одном месте. Время и кнопку он проверяет между обслуживанием обмена.

Вопросы 41-50. Буферы и строки

41. Зачем нужны RX и TX?

RX временно хранит принятые байты. TX хранит байты, которые ещё нужно отправить. Они помогают основному коду и UART работать в своём темпе.

42. Каковы размеры буферов?

По умолчанию RX - 128 байт, TX - 512 байт. Это параметры сборки, а не бесконечные очереди.

43. Что значит «кольцевой буфер»?

После последней ячейки используется первая свободная. Данные остаются в порядке очереди, а весь массив не нужно сдвигать.

44. Что означает успешный serial_write?

Весь блок принят в очередь TX. Это ещё не подтверждение, что байты уже дошли до компьютера.

45. Что будет, если в TX не хватает места?

serial_write вернёт false и не добавит даже часть блока. Консоль заранее проверяет место под эхо и ответ.

46. Сколько символов может быть в команде?

До 63. В буфере из 64 ячеек ещё одна нужна для нуля, обозначающего конец строки в C.

47. Что делают Backspace и Delete?

Убирают последний символ из собранной команды. Внешний вид удаления в терминале может отличаться от изменения буфера.

48. Что такое парсинг команды?

Проверка текста: какая команда введена и допустим ли её аргумент. Например, после set timeout должны идти только цифры разрешённого числа.

49. Что будет при потере входных данных?

RX очищается, а текущая строка отвергается до Enter. Программа просит повторить команду, чтобы не выполнить повреждённый текст.

50. Может ли очередь гарантировать приём бесконечного потока?

Нет. Место в памяти ограничено, flow control здесь не используется. При слишком быстром вводе возможна потеря данных.

Вопросы 51-60. Флаги и регистры

Сначала смысл флага, затем его техническое название.

51. Что означает RXNE?

Пришёл байт, который ещё не прочитан. Его можно забрать из регистра DR.

52. Что означает TXE?

Регистр передачи DR готов принять байт. Это не означает, что предыдущий кадр уже целиком ушёл по проводу.

53. Что означает TC?

Передача последнего кадра закончилась. Этот флаг нужен, чтобы убедиться: оставшихся битов на линии уже нет.

54. Зачем ждать TC перед сменой I/P?

Чтобы закончить уже начатую передачу перед перенастройкой UART. Одного пустого TX-буфера или TXE для этого недостаточно.

55. Что хранит SR?

Флаги состояния и ошибок USART. По ним драйвер понимает, что можно сделать сейчас.

56. Зачем нужен DR?

Через него читают принятый байт и записывают байт для передачи. Под одним адресом аппаратно обслуживаются разные направления.

57. Зачем нужен BRR?

Он задаёт делитель скорости. Здесь при тактировании USART1 16 МГц в него записывается 278, то есть 0x116.

58. Для чего CR1, CR2 и CR3?

Они хранят настройки USART: включение приёмника и передатчика, формат кадра и разрешения нужных событий.

59. Что такое ORE?

Ошибка переполнения приёмника: новый байт пришёл, когда предыдущий ещё не забрали из аппаратного регистра. Это не то же самое, что полный программный RX-буфер.

60. Как здесь снимаются ошибки приёма?

Для этого USART читается SR, затем DR. Нельзя заменять эту последовательность произвольным SR |=. Чтение DR также убирает RXNE.

Вопросы 61-70. HAL и собственный драйвер

61. Чем отличаются две сборки?

Одна управляет UART через HAL, другая - собственным кодом через регистры. Команды, светофор и очереди в них общие.

62. Можно ли выбрать HAL командой из терминала?

Нет. Это выбор до сборки и прошивки. Команды on/off меняют только способ обмена I/P.

63. Собственная сборка тоже поддерживает I и P?

Да. Опрос и прерывания реализованы в обеих версиях, и между ними можно переключаться во время работы.

64. Какие HAL-функции используются для UART с прерываниями?

HAL_UART_Receive_IT запускает приём, HAL_UART_Transmit_IT - отправку. После события HAL вызывает специальные функции уведомления - callbacks.

65. Что такое callback?

Функция, которую библиотека вызывает при событии. Например, после приёма байта callback кладёт его в RX и снова запускает приём.

66. Создаёт ли HAL_UART_Receive_IT очередь на 128 байт?

Нет. В этой реализации запускается приём одного байта, а очередь написана отдельно в serial.c.

67. Зачем передаваемый байт хранится в постоянной переменной?

Передача с прерываниями закончится позже вызова функции. Память с байтом должна оставаться доступной всё это время. Здесь используется статическая tx_byte.

68. Что означает busy?

Что байт ещё передаётся. В режиме I флаг остаётся установленным до завершения по TC, чтобы main не заменил используемый байт раньше времени.

69. Можно ли в собственной версии вообще не использовать HAL?

По заданному решению собственный именно UART-драйвер. HAL остаётся для системных часов и SysTick, а HAL/LL UART в этой сборке не вызываются.

70. Что отличает собственный драйвер от LL?

LL тоже даёт функции близко к регистрам. Здесь сам алгоритм приёма, отправки и обработки событий написан вручную без HAL/LL UART.

Вопросы 71-80. Детали, которые могут спросить

71. Что делать после UART-ошибки FE, NE или PE?

FE означает неправильное окончание кадра, NE - обнаруженный шум, PE - ошибку чётности. Код помечает ввод как потерянный и не выполняет текущую строку; в текущем 8N1 чётность выключена.

72. Как основной код и прерывание делят RX?

Оба работают с общими данными, поэтому чтение байта защищено очень коротким запретом прерываний. Прежнее состояние запрета затем восстанавливается.

73. Зачем сохранять PRIMASK, а не просто разрешить прерывания?

PRIMASK хранит состояние общего запрета обычных прерываний. Если они уже были запрещены до функции, она не должна самовольно включать их при выходе.

74. Volatile решает все проблемы общего доступа?

Нет. Он нужен для правильных обращений компилятора к изменяемым данным, но несколько действий подряд не становятся от этого неделимыми. Поэтому нужны короткие защищённые участки.

75. Почему за один вызов консоли разбирают не больше 16 байт?

Чтобы большой ввод не занял весь основной цикл. После порции символов программа возвращается к кнопке и светофору.

76. Как программа защищена от слишком большого timeout?

Разборщик проверяет границу до очередного умножения числа на 10. Поэтому огромное число не переполнится и не превратится случайно в допустимое маленькое.

77. Что делает set interrupts с уже принятой очередью?

RX-очередь сохраняется. Переключение ждёт конца передачи, затем меняет настройки и разрешения UART. После этого продолжается разбор команд.

78. Чьим способом отправится OK после переключения?

Уже новым. Сначала заканчивается старый обмен и применяется I/P, затем консоль ставит OK в очередь.

79. Что проверяют тесты?

Обе версии UART, команды, ошибки, переполнения, очереди, переключение I/P и запуск прошивки. Модель исполняет ARM-код, но не проверяет реальный USB и электрические сигналы.

80. Что смотреть, если нет связи?

Питание и DBG USB, правильный COM-порт, 57600 8N1 и драйвер Interface 1. Затем в коде - lab2_fault, настройки USART1 и факт выполнения main.

Практика 1. Команды и ожидаемые ответы

В таблице показаны ответы без повторённого платой текста команды - эхо на экране будет отдельно.

Ввод и Enter	Ожидаемый результат
?	Например: red, mode 1, timeout 12, I. Фаза зависит от момента запроса.
set timeout 8	OK. Обычный красный станет 8 с; от кнопки может сократиться до 2 с.
set mode 2	OK. Кнопка больше не сокращает красный; старая просьба сброшена.
set mode 1	OK. Новые нажатия снова учитываются.
set interrupts off	OK. Следующий ? должен показать P.
set interrupts on	OK. Следующий ? должен показать I.
set timeout 0	Ошибка. Ноль запрещён.
set timeout 86401	Ошибка. Больше одних суток запрещено.
set interrupts ON	Ошибка. Регистр букв значим.

Сообщения об ошибке в программе

```
ERROR: unsupported command or invalid argument
ERROR: input lost or line too long; repeat command
```

Первое означает неизвестную команду или неверный аргумент. Второе - потерю ввода либо слишком длинную строку. После восстановления отправь команду заново.

Задача

Можно ли принять `set timeout 00012`? Да: это число 12. Можно ли принять `set timeout +12`? Нет: знак плюс запрещён, разрешены только цифры. Лишний пробел после числа тоже даёт ошибку.

Практика 2. Что произойдёт со светофором

Ниже время красного считается от начала текущей фазы. Кнопка даёт событие после проверки устойчивого нажатия в течение 30 мс, как в ЛР 1.

Ситуация	Что должно произойти
Timeout 8; нажатие в mode 1 подтвердилось на 1-й секунде	Зелёный на отметке 2 с. Осталось примерно 1 с.
Timeout 8; нажатие подтвердилось на 5-й секунде	Зелёный сразу после подтверждения: 2 с уже прошли.
Красный идёт 5 с; команда set timeout 4	При ближайшей проверке будет переход: новый порог уже достигнут.
Красный идёт 5 с; set timeout 20; просьбы от кнопки нет	Начало не меняется. До порога остаётся около 15 с.
Запрос от кнопки сохранён; затем set mode 2	Запрос стирается, используется обычная длительность красного.
В mode 2 кнопку держат; затем set mode 1	Автоматического нового нажатия нет: нужно отпустить и нажать.
Во время мигания лампа сейчас выключена; отправили ?	Фаза в ответе всё равно blinking green.

Что здесь работает так же, как в ЛР 1

Обычный зелёный игнорирует новые нажатия. Мигание и жёлтый сохраняют просьбу на ближайший красный, если кнопка разрешена. Удержание даёт одно событие. Время отсчитывается без длинных пауз в основном цикле.

Вопрос с подвохом

Мы выключили прерывания UART. Должен ли замереть светофор? Нет: SysTick продолжает работать, а основной цикл теперь сам опрашивает UART.

Что показать на защите

Короткая последовательность

1. Связь. Подключить терминал с настройками 57600 8N1, показать приветствие, эхо и ответ на ?.

2. Кнопка. Выполнить `set mode 2`, затем `set mode 1`. Объяснить, что отключается именно влияние кнопки.

3. Время. Выполнить `set timeout 8`. На следующей красной фазе показать 8 с без запроса и сокращение до 2 с с запросом.

4. UART. Выполнить `set interrupts off`, проверить P командой ?. Затем `set interrupts on`, проверить I. Объяснить, почему светофор продолжает работать.

5. Ошибки. Отправить `set timeout 0` и показать отказ. Затем правильную команду: обмен после ошибки должен работать.

6. Свой драйвер. Прошить вторую сборку и показать те же команды. Объяснить, что поменялся код управления UART, а не правила светофора.

Как собрать из корня проекта

```
python3 lab2/tools/build.py
python3 lab2/tools/build.py --backend register
```

Результаты: `lab2/build/lab2_hal.elf` и `lab2/build/lab2_register.elf`. Для прошивки используется соответствующий ELF. В CubeIDE выбор собственного драйвера задаёт символ `UART_REGISTER_BACKEND`.

Пять фраз перед защитой

UART передаёт байты. Очереди дают им подождать. Enter завершает команду. I/P выбирает способ обслуживания UART. Mode 1/2 выбирает влияние кнопки.

Затем повтори главное различие флагов: TXE - можно положить байт, TC - передача последнего кадра завершилась. Измерять секунды нужно при свободном выполнении, без остановок отладчика.

Карта проекта и проверка

Что искать	Файл внутри lab2
Пины и скорость UART	Core/Inc/board.h, Core/Src/uart_pins.c
Главный цикл и начальные настройки	Core/Src/main.c
Очереди и смена I/P	Core/Src/serial.c
Эхо, строка, ответы и восстановление	Core/Src/console.c
Допустимые команды и числа	Core/Src/command.c
Режим кнопки и длительность красного	Core/Src/traffic.c
Версия HAL / собственная версия	Core/Src/uart_hal.c / uart_register.c
Программные проверки	tests/test_lab2.c, tests/run_tests.py

Что проверено при подготовке

03.11.2026 обе прошивки и обе тестовые сборки собраны в отдельной временной копии проекта. Для HAL прошли 56 проверок C и 105 проверок обмена. Для собственного драйвера - 56 проверок C и 101 проверка обмена.

Тесты также проверили запуск готовых прошивок от Reset_Handler и приветствие. Это подтверждает работу кода в программной модели. Физическая связь USB/COM и работа реального стенда в рамках этой подготовки не проверялись.

По каким материалам составлена памятка

Основные источники - исходники lab2, README.md, docs/DEFENSE.md, docs/VALIDATION.md и схема docs/SDK1.1M.pdf. Раздел 5.2 пособия ИТМО использован для сверки смысла задания. Подробные решения, размеры очередей и ограничения описывают именно код этого проекта.

Пособие ИТМО: раздел 5.2, UART

RM0090: регистры STM32F4, раздел USART

Исходники HAL UART от STMicroelectronics, версия 1.8.1